

tf.unstack Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/unstack

Unpacks the given dimension of a rank- R tensor into rank- $(R-1)$ tensors.

Function Signatures

($R-1$)

```
tf.unstack( value, num=None, axis=0, name='unstack' )
```

Code Examples

```
tf.unstack(  
value, num=None, axis=0, name='unstack'  
)
```

```
tf.unstack(  
value, num=None, axis=0, name='unstack'  
)
```

```
x = tf.reshape(tf.range(12), (3,4))  
p, q, r = tf.unstack(x)  
p.shape.as_list()  
[4]
```

```
x = tf.reshape(tf.range(12), (3,4))
```

```
p, q, r = tf.unstack(x)
```

```
p.shape.as_list()
```

Documentation

Unpacks the given dimension of a rank-R tensor into rank-(R-1) tensors.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.unstack`

Used in the notebooks

- Scalable model compression

Unpacks tensors from value by chipping it along the axis dimension.

This is the opposite of `stack`.

More generally if you have a tensor of shape (A, B, C, D):

The number of tensor returned is equal to the length of the target axis:

The shape of each result tensor is equal to the shape of the input tensor,
with the target axis removed.

The value of each tensor `items[i]` is equal to the slice of input across
axis at index `i`:

Python iterable unpacking

With eager execution you can unstack the 0th axis of a tensor using python's
iterable unpacking:

`unstack` is still necessary because Iterable unpacking doesn't work in
a `@tf.function`: Symbolic tensors are not iterable.

You need to use `tf.unstack` here:

Unknown shapes

Eager tensors have concrete values, so their shape is always known. Inside a `tf.function` the symbolic tensors may have unknown shapes. If the length of axis is unknown `tf.unstack` will fail because it cannot handle an unknown number of tensors:

If you know the axis length you can pass it as the `num` argument. But this must be a constant value.

If you actually need a variable number of tensors in a single `tf.function` trace, you will need to use explicit loops and a `tf.TensorArray` instead.

Returns

Raises